

INVENTORY MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

ROHAN KUMAR PRADHAN (250320100083)

in partial fulfilment for the award of the degree of

MASTER OF COMPUTER APPLICATION



Centurion
UNIVERSITY

Shaping Lives...
Empowering Communities...

SCHOOL OF ENGINEERING AND TECHNOLOGY

BHUBANESWAR CAMPUS

CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT

ODISHA

APRIL 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF ENGINEERING AND TECHNOLOGY

BHUBANESWAR CAMPUS

Bonafide Certificate

Certified that this project report on “**INVENTORY MANAGEMENT SYSTEM**” is the Bonafide work of “**Rohan Kumar Pradhan**” who carried out the project work under my supervision. This is to further certify to the best of my knowledge, that this project has not been carried out earlier in this institute and the university.

SIGNATURE

(Mr. Saroj Kumar Sahoo)

Professor of Computer Science and Engineering

Certified that the above-mentioned project has been duly carried out as per the norms of the college and statutes of the university.

SIGNATURE

(Mr. Rakesh Kumar Ray)

HEAD OF THE DEPARTMENT

Professor of Computer Science and Engineering

DEPARTMENT SEAL

Declaration

I hereby declare that the project entitled “**Inventory Management System**” submitted for the “**Java Programming**” of 2nd semester MCA in School of Computer Science and Engineering is my original work and the project has not formed the basis for the award of any Degree / Diploma or any other similar titles in any other University / Institute.

Name of the Student: Rohan Kumar Pradhan

Signature of the Student:

Registration No: 250320100083

Place: Bhubaneswar

Date:

Acknowledgement

I wish to express my profound and sincere gratitude to **Mr. Saroj Kumar Sahoo**, Department of Computer Science and Engineering, SoET, Bhubaneswar Campus, who guided me into the intricacies of this project nonchalantly with matchless magnanimity.

I thank **Prof. Rakesh Kumar Ray**, Head of the Dept. of Department of Computer Science and Engineering, SoET, Bhubaneswar Campus and **Dr. Sujata Chakravarty**, Dean, School of Engineering and Technology, Bhubaneswar Campus for extending their support during Course of this investigation.

I would be failing in my duty if I don't acknowledge the cooperation rendered during various stages of image interpretation by **Prof. Saroj Kumar Sahoo**

I am highly grateful to Prof. **Saroj Kumar Sahoo** who evinced keen interest and invaluable support in the progress and successful completion of my project work.

I am indebted to **Prof. Saroj Kumar Sahoo** for their constant encouragement, co-operation and help. Words of gratitude are not enough to describe the accommodation and fortitude which they have shown throughout my endeavor.

Name of the Student: Rohan Kumar Pradhan

Signature of the Student:

Registration No: 250320100083

Place: Bhubaneswar

Date:

TABLE OF CONTENTS

Certificate	i
Acknowledgement	ii
Table of Contents	iii
Abstract	iv
Chapter 1: Introduction	1
1.1 Overview of Inventory Management Systems	1
1.2 Need for Inventory Management	2
1.3 Problem Statement	3
1.4 Objectives of the Project	4
1.5 Scope of the System	5
Chapter 2: Literature Review	7
2.1 Existing Inventory Management Systems	7
2.2 Limitations of Traditional Systems	8
2.3 Need for Digital Inventory Systems	9
2.4 Comparative Analysis of Existing Solutions	10
Chapter 3: System Analysis	12
3.1 Functional Requirements	12
3.2 Non-Functional Requirements	13
3.3 Feasibility Study	14
3.4 System Architecture	16
Chapter 4: System Design	18
4.1 Database Design	18
4.2 Entity-Relationship Diagram	19
4.3 Data Flow Diagram	20
4.4 UML Diagrams	21
Chapter 5: Implementation	23
5.1 Login Module	23
5.2 Registration Module	24
5.3 Admin Dashboard	24
5.4 User Dashboard	25
5.5 Product Management	25
5.6 Request Management System	26
5.7 Stock Update Logic	27
5.8 Status Tracking System	27
5.9 Billing System with GST	28
5.10 Invoice Generation	28
5.11 User Management	29
5.12 Password Reset Module	29

Chapter 6: Testing	31
6.1 Test Cases	31
6.2 Input/Output Validation	33
6.3 Error Handling	34
6.4 Debugging Issues Faced	35
Chapter 7: Results and Discussion	37
7.1 Output Descriptions	37
7.2 Performance Analysis	39
7.3 Observations	40
Chapter 8: Conclusion	46
8.1 Summary of the Project	46
8.2 Learning Outcomes	47
8.3 Final Remarks	48
References	49

ABSTRACT

The Inventory Management System (IMS) presented in this report is a comprehensive desktop application developed using Java Swing for the graphical user interface (GUI) and MySQL as the back-end relational database management system. The primary objective of this project is to automate and streamline the management of product inventories, user accounts, stock levels, requests, billing, and invoicing within a small-to-medium enterprise setting. The system is designed to eliminate the inefficiencies and inaccuracies associated with manual, paper-based inventory tracking systems, replacing them with a reliable, real-time, and digitally integrated solution.

The system supports two distinct user roles: Administrator and Regular User. Administrators are empowered with full control over the system, including product management (add, update, delete, search), user account management, request approval or rejection, stock updates, and report generation. Regular users can browse available products, submit requests for items, track the status of their requests, and generate invoices for fulfilled requests. The billing module incorporates GST (Goods and Services Tax) calculation with a configurable tax rate, enabling the generation of professional invoices in a pop-up format within the application itself.

The system architecture follows a three-tier model comprising the presentation layer (Java Swing UI), the business logic layer (Java classes and controllers), and the data layer (MySQL database accessed via JDBC). The database design includes well-normalized tables for Users, Products, and Requests, ensuring data integrity, minimal redundancy, and efficient query processing. Security features include role-based access control, password hashing for stored credentials, and administrative password reset capabilities.

The development of this system involved extensive use of object-oriented programming principles including encapsulation, inheritance, polymorphism, and abstraction. Java's rich standard library, combined with the powerful querying capabilities of MySQL, facilitated the development of a robust and maintainable codebase. The application was thoroughly tested using black-box and white-box testing techniques, and several debugging sessions were conducted to resolve logical errors, database connectivity issues, and UI rendering inconsistencies.

The results demonstrate that the system is capable of handling a realistic volume of product records and user transactions with satisfactory performance. The user interface has been designed with usability in mind, ensuring ease of navigation and minimal training requirements for end users. The project successfully achieves its stated objectives and lays a solid foundation for future enhancements such as web-based deployment, cloud database integration, advanced reporting modules, and enhanced security features. This report documents all phases of the software development lifecycle, from requirements analysis and system design to implementation, testing, and future scope.

Keywords: Inventory Management System, Java Swing, MySQL, JDBC, CRUD Operations, Billing, GST, Role-Based Access Control, Desktop Application, Stock Management.

CHAPTER 1: INTRODUCTION & PROJECT OVERVIEW

1.1 Overview of Inventory Management Systems

Inventory management is one of the most critical operational functions within any business enterprise, regardless of its size or the nature of the products it handles. At its core, inventory management refers to the systematic process of ordering, storing, tracking, and controlling a company's stocked goods. These goods may include raw materials used in manufacturing, partially assembled goods, finished products awaiting dispatch, or supplies used internally within an organization. The primary goal of an effective inventory management system is to ensure that the right quantity of the right item is available at the right time, thereby minimizing costs while maximizing operational efficiency.

Historically, inventory management was carried out manually, relying on handwritten ledgers, physical stocktaking, and paper-based requisition forms. While such methods were adequate in an era of simpler supply chains and smaller product catalogues, they are wholly inadequate for modern businesses that deal with large volumes of products, complex supplier relationships, and dynamic demand patterns. The advent of computing technology in the latter half of the twentieth century paved the way for digital inventory management systems, which began as simple spreadsheet-based solutions and have since evolved into sophisticated enterprise resource planning (ERP) systems capable of integrating inventory management with procurement, sales, accounting, and customer relationship management.

An Inventory Management System (IMS) is a software application designed to track inventory levels, orders, sales, and deliveries in real time. Modern inventory management systems can be broadly categorized into standalone desktop applications, web-based systems, and cloud-native solutions. Each category offers distinct advantages and trade-offs in terms of cost, scalability, accessibility, and complexity. Standalone desktop applications, such as the one developed in this project, are particularly suitable for small and medium-sized enterprises (SMEs) that require a reliable, cost-effective solution without the overhead of web infrastructure or cloud services.

The system described in this report is a desktop-based Inventory Management System developed using Java Swing, a platform-independent GUI toolkit that is part of Java's Standard Edition. Java Swing provides a rich set of graphical components including windows, panels,

buttons, tables, dialogs, and menus, enabling the development of professional and responsive user interfaces. The back-end data storage is handled by MySQL, an open-source relational database management system widely used in both academic and industrial settings for its reliability, performance, and ease of use. The two technologies are integrated using the Java Database Connectivity (JDBC) API, which provides a standard interface for Java applications to interact with relational databases.

1.2 Need for Inventory Management

The need for an effective inventory management system arises from the fundamental challenges associated with managing physical goods in a dynamic business environment. Without a proper system in place, organizations frequently encounter problems such as overstocking, understocking, stock discrepancies, misplaced items, delayed order fulfillment, and financial losses due to spoilage or obsolescence. These problems are not merely operational inconveniences; they can have significant financial implications and can negatively impact customer satisfaction, supplier relationships, and overall business reputation.

Overstocking occurs when an organization maintains excess inventory beyond its immediate requirements. This leads to increased holding costs, including warehouse space, insurance, and the risk of product deterioration or obsolescence. Conversely, understocking or stockouts occur when inventory levels fall below the minimum required to fulfill current demand, resulting in missed sales opportunities, disrupted production schedules, and dissatisfied customers. Striking the optimal balance between these two extremes requires real-time visibility into inventory levels, accurate demand forecasting, and timely replenishment triggers — all of which are capabilities that a well-designed inventory management system provides.

In educational institutions and administrative offices, which represent another significant application domain for this system, inventory management encompasses the tracking of stationery, electronic equipment, laboratory supplies, and other consumables. Requisition management — the process by which users formally request items from the inventory — is a particularly important sub-function in such environments, where multiple users may be competing for limited resources and where accountability and auditability are paramount. Without a digital system, tracking who requested what, when the request was submitted,

whether it was approved, and whether the items were actually dispensed becomes an unwieldy administrative challenge.

The financial dimension of inventory management is equally compelling. Inventory often represents one of the largest assets on a company's balance sheet, and inefficiencies in its management directly translate into reduced profitability. An automated inventory management system that provides accurate stock valuation, tracks the cost of goods sold, and supports billing with tax calculations such as GST can significantly reduce the manual accounting burden and improve the accuracy of financial reporting. The Billing System with GST Calculation incorporated in this project addresses precisely this need, enabling the system to generate tax-compliant invoices automatically.

1.3 Problem Statement

Despite the widespread availability of inventory management solutions in the market, a significant proportion of small and medium-sized enterprises, educational institutions, government offices, and non-profit organizations continue to rely on outdated manual methods or simple spreadsheet-based approaches for managing their inventories. This reliance on primitive methods stems from a combination of factors including high software licensing costs, complex setup requirements, steep learning curves associated with commercial ERP systems, and a general lack of awareness about affordable, easy-to-use alternatives.

Manual inventory management systems are inherently prone to human error. Data entry mistakes, miscalculations, and omissions can result in inaccurate stock records, which in turn lead to poor purchasing decisions and operational disruptions. Furthermore, paper-based systems offer no mechanism for real-time monitoring of stock levels, making it impossible for managers to receive timely alerts when stock falls below critical thresholds. The absence of a centralized, searchable database means that retrieving historical records is time-consuming and often unreliable.

The specific problem addressed by this project is the absence of an integrated, role-based, desktop inventory management system that supports not only product tracking and CRUD (Create, Read, Update, Delete) operations, but also requisition management, billing with GST computation, invoice generation, and user management — all within a single, cohesive application. The target users are administrators responsible for managing the inventory and regular users who need to request products and track the status of their requests. The system

must be easy to use, secure, reliable, and deployable on standard Windows or Linux desktop computers without requiring specialized infrastructure.

This project therefore undertakes the design and development of such a system from the ground up, applying established software engineering principles and best practices in Java programming and relational database design to deliver a functional, well-tested, and thoroughly documented solution that meets the identified needs.

1.4 Objectives of the Project

The primary objective of this project is to design, develop, and test a fully functional desktop-based Inventory Management System that addresses the shortcomings of manual and spreadsheet-based approaches. This overarching objective encompasses several specific sub-objectives, each of which contributes to the completeness and quality of the final system.

The first specific objective is to develop a secure user authentication system that supports two roles: Administrator and Regular User. The authentication module must validate user credentials against securely stored records in the MySQL database and ensure that each user is presented with an interface appropriate to their role upon successful login. The system must also provide a registration facility for new users and an administrative password reset capability.

The second objective is to implement comprehensive product management functionality, enabling administrators to add new products to the inventory, update existing product details such as name, category, description, price, and quantity, delete products that are no longer relevant, and search for products using various criteria. This CRUD functionality must be reflected immediately in the database and in the user interface's product listing table.

The third objective is to develop a request management system through which regular users can browse available products, submit requests for specific quantities of products, and monitor the status of their requests. Administrators must be able to view all pending requests, approve or reject them, and record the reasons for rejections. The stock quantity must be updated automatically upon approval of a request to reflect the reduction in available inventory.

The fourth objective is to implement a billing system that computes the total cost of approved requests, including applicable GST at a configurable rate, and generates a professional invoice. The invoice generation module must produce a pop-up display containing all relevant details including product name, quantity, unit price, GST amount, and grand total.

The fifth and final objective is to thoroughly test the system to ensure correctness, robustness, and usability, documenting test cases, observed behaviours, and any limitations identified during the testing phase.

1.5 Scope of the System

The scope of the Inventory Management System developed in this project is defined both in terms of the functionalities it includes and the context in which it is intended to be deployed. Understanding the scope is important both for managing expectations and for providing a clear framework within which future enhancements can be evaluated.

In terms of functional scope, the system covers user registration and authentication, role-based access control for administrators and regular users, complete product lifecycle management, stock tracking and real-time quantity updates, a multi-step request management workflow, GST-inclusive billing, and invoice generation. The system is designed to handle a moderate volume of users and products — sufficient for departmental or institutional use — and has been tested with realistic datasets to validate its performance.

In terms of deployment scope, the system is a standalone desktop application designed to run on any computer that has the Java Runtime Environment (JRE) installed, version 8 or higher. The MySQL database server must be installed and accessible on the same machine or on a local network. The system does not require an internet connection for its core functionality, making it suitable for deployment in environments with limited or no internet access. It does not include any web-based or mobile interface; such extensions are identified as potential future enhancements.

The system does not include advanced features such as automated purchase order generation, supplier management, demand forecasting, barcode scanning, or integration with third-party accounting software. These features, while valuable in an enterprise context, are beyond the scope of this academic project and are discussed in the future enhancements section. The system also does not include an email notification or OTP verification module.

Overall, the system is intended to serve as a fully functional, production-ready prototype that demonstrates the feasibility and effectiveness of a Java Swing and MySQL-based inventory management solution, and that provides a solid basis for further development and expansion.

CHAPTER 2: LITERATURE REVIEW

2.1 Existing Inventory Management Systems

A substantial body of literature and a wide variety of commercial and open-source software products exist in the domain of inventory management. Reviewing these existing systems provides valuable context for understanding the current state of the field, the technological approaches that have been employed, and the gaps that remain unaddressed — particularly for small and medium-scale deployments with limited budgets and technical resources.

At the high end of the market, enterprise resource planning (ERP) systems such as SAP ERP, Oracle NetSuite, and Microsoft Dynamics 365 provide comprehensive inventory management capabilities as part of broader integrated business management suites. These systems offer sophisticated features including multi-warehouse management, demand forecasting, automated reorder points, integration with supply chain and procurement modules, and detailed analytics dashboards. However, these systems are characteristically expensive, both in terms of licensing costs and implementation effort. They require dedicated IT professionals for setup, customization, and maintenance, and their complexity makes them unsuitable for organizations with straightforward inventory management needs.

In the mid-market segment, solutions such as Zoho Inventory, inFlow Inventory, and Fishbowl Inventory offer a more accessible range of features at a lower cost. These cloud-based or on-premise applications provide core inventory management functionality — product tracking, order management, reporting, and invoicing — with more user-friendly interfaces than enterprise ERP systems. However, even these solutions typically involve recurring subscription fees, require stable internet connectivity for cloud-based variants, and may include features far in excess of what is needed by a small departmental unit or educational institution.

Open-source inventory management systems such as openBoxes, Odoo Community Edition, and PartKeepr represent a cost-free alternative, but they typically require significant technical expertise to install, configure, and maintain. Most of these systems are web-based, requiring a web server, a database server, and often additional dependencies, which adds infrastructure complexity. Furthermore, the documentation and community support for open-source solutions can be inconsistent, posing challenges for non-technical users.

Academic literature on inventory management systems reveals a rich history of research into optimization algorithms, demand forecasting models, and supply chain integration frameworks. Seminal works such as Wilson's economic order quantity (EOQ) model and the ABC analysis methodology for inventory classification have been extensively studied and implemented in software systems. More recent research has focused on applying machine learning to demand forecasting, blockchain technology to supply chain traceability, and IoT sensors to real-time inventory monitoring. While these advanced approaches are beyond the scope of the current project, they represent important directions for the future evolution of inventory management systems.

2.2 Limitations of Traditional Systems

Traditional inventory management systems, particularly those based on manual record-keeping and simple spreadsheet tools, suffer from a range of fundamental limitations that render them inadequate for modern business environments. Understanding these limitations is essential for appreciating the value that a digital, database-backed system such as the one developed in this project provides.

The most pervasive limitation of manual systems is their inherent susceptibility to human error. Every stage of a manual inventory process — from physically counting stock to recording quantities in a ledger, transcribing data onto order forms, and computing totals — introduces opportunities for mistakes. A single transposition error in a stock count can propagate through the entire system, resulting in cascading inaccuracies in stock valuations, order quantities, and financial reports. Studies in operations management have consistently found that manual data entry error rates range from one to four percent, which, while seemingly small, can translate into significant financial losses in high-volume inventory environments.

Scalability is another critical limitation of traditional systems. As an organization grows and its product catalogue expands, manual systems become increasingly unwieldy. Managing thousands of SKUs (stock keeping units) using paper records or even basic spreadsheets is practically infeasible without a substantial investment in administrative labour. The time required to perform a physical stocktake grows proportionally with the size of the inventory, meaning that large organizations may only be able to perform full audits infrequently, leaving them exposed to prolonged periods of inaccurate stock data.

Concurrency and multi-user access represent further limitations. In environments where multiple staff members need to access and update inventory records simultaneously, manual systems provide no mechanism for conflict resolution. Two users updating the same record at the same time — a common scenario in busy warehouses or offices — can result in data corruption or loss. Spreadsheet-based systems, while better than paper records, suffer from similar limitations in multi-user environments, particularly when files are shared over network drives without robust version control.

Security and auditability are also significant concerns with traditional systems. Paper records can be lost, damaged, or tampered with, and spreadsheets are easily modified without any audit trail. In contrast, a well-designed database-backed system can record every transaction with timestamps and user identifiers, enabling comprehensive audit trails that support accountability and fraud prevention.

2.3 Need for Digital Inventory Systems

The transition from manual to digital inventory management systems is driven by a confluence of technological advancements, changing business requirements, and increasing regulatory demands. Digital inventory management systems address the fundamental limitations of their manual counterparts by leveraging the speed, accuracy, and data integrity guarantees that modern database technologies provide.

Real-time data availability is perhaps the most compelling advantage of digital inventory systems. When a product is added, updated, or removed from the inventory, the change is immediately reflected in the database and is visible to all authorized users simultaneously. This real-time visibility enables managers to make informed decisions based on current stock levels rather than data that may be days or weeks out of date. For businesses where stockouts can result in lost sales or operational disruptions, this real-time awareness is invaluable.

Digital systems also provide powerful search and filtering capabilities that are simply not feasible in manual environments. A user can instantly retrieve all products belonging to a specific category, all products with stock below a specified threshold, or all requests submitted by a particular user within a given date range. These queries, which would require hours of manual searching in a paper-based system, can be executed in milliseconds by a properly indexed relational database.

The need for GST compliance and accurate financial reporting has further accelerated the adoption of digital inventory systems in India and many other countries. Since the introduction of the Goods and Services Tax in India in 2017, businesses are required to maintain accurate records of all taxable transactions and to issue tax-compliant invoices. Digital inventory systems that incorporate GST calculation and invoice generation directly address this regulatory requirement, reducing the risk of non-compliance and the associated penalties.

From an educational perspective, the development of a digital inventory management system provides computer science students with a valuable opportunity to apply and integrate knowledge from multiple academic disciplines, including database management, software engineering, object-oriented programming, and human-computer interaction. The project develops practical skills that are directly transferable to professional software development environments.

2.4 Comparative Analysis of Existing Solutions

A systematic comparison of existing inventory management solutions reveals significant variations in functionality, cost, complexity, and suitability for different use cases. This comparative analysis situates the system developed in this project within the broader landscape of available solutions and clarifies the specific niche it is designed to occupy.

Commercial ERP systems such as SAP and Oracle offer the most comprehensive feature sets but at the highest cost and complexity. They are clearly designed for large enterprises with dedicated IT teams and are not appropriate for the target audience of this project. Mid-market SaaS solutions like Zoho Inventory are more accessible but still require internet connectivity and recurring subscription fees, making them unsuitable for organizations with limited connectivity or budgets. Furthermore, most SaaS solutions do not offer the degree of customization that may be required for institution-specific workflows.

Desktop applications built using technologies such as Java, C#, or Python with SQLite or MySQL backends represent the most appropriate category for the target deployment environment. Compared to commercially available desktop inventory tools, the system developed in this project offers several specific advantages: it is entirely open-source and free to use, it is tailored to the specific workflow of role-based requisition management, it includes an integrated billing system with GST calculation, and its entire codebase and database schema are fully transparent and modifiable.

In terms of specific technological choices, the use of Java Swing for the GUI ensures platform independence, meaning the application can run on any operating system with a compatible JRE. This contrasts with Windows-only solutions built using C# and WinForms, which are limited to the Windows ecosystem. MySQL's robust performance, extensive community support, and widespread use in both academic and professional settings make it a logical choice for the database layer. The JDBC API provides a well-standardized and thoroughly documented mechanism for connecting the two components.

Based on this comparative analysis, it is evident that while numerous inventory management solutions exist, there remains a clear need for simple, free, customizable, role-based desktop applications that can be deployed in resource-constrained environments. The system developed in this project directly addresses this need, offering a functional and well-documented solution that balances simplicity with a comprehensive feature set.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Functional Requirements

Functional requirements define the specific behaviours and functions that the Inventory Management System must perform. They describe what the system does rather than how it does it, and they form the primary basis for system design, implementation, and testing. The functional requirements for this system have been identified through a careful analysis of the intended use cases and the stakeholder needs of both administrators and regular users.

The user authentication requirements specify that the system must provide a login interface where users can enter their username and password to access the system. The system must validate the provided credentials against the stored records in the Users table and authenticate the user only if a matching, active account is found. Upon successful authentication, the system must determine the user's role — either Administrator or Regular User — and navigate to the appropriate dashboard. The system must prevent unauthorized access by rejecting invalid credentials and displaying an informative error message. The system must also provide a new user registration form that collects the necessary account information and creates a new user record in the database.

The product management requirements specify that the system must allow administrators to add new products by entering details such as product name, category, description, unit price, and initial stock quantity. The system must allow administrators to update any of these details for existing products and to delete products from the system. The system must provide a product listing view that displays all current products in a tabular format with search and filter capabilities. All product changes must be immediately persisted to the database and reflected in the UI.

The request management requirements specify that regular users must be able to browse the product listing, select a product, specify a requested quantity, and submit a request. The system must validate that the requested quantity does not exceed the available stock before submission. Administrators must be able to view a list of all pending requests, approve requests (which triggers a corresponding reduction in stock quantity), or reject requests with an optional reason. Both administrators and users must be able to view the current status of each request — Pending, Approved, or Rejected.

The billing and invoicing requirements specify that the system must be able to compute the total cost of an approved request based on the unit price of the product and the approved quantity. The system must apply GST at a configurable percentage rate to compute the tax amount and the grand total. The system must generate a formatted invoice in a pop-up dialog that can be reviewed by the user. User management requirements specify that administrators must be able to view a list of all registered users, deactivate or delete user accounts, and reset user passwords.

3.2 Non-Functional Requirements

- Define system quality, performance, and operational standards (how well the system works).

Performance:

- Response time should be under 2 seconds for user actions.
- Handle database queries up to 1000 records efficiently.
- No noticeable lag during normal operations.

Security:

- Store passwords using hashing (bcrypt / SHA-256 preferred).
- Enforce role-based access control.
- Validate inputs to prevent SQL injection attacks.

Reliability:

- Handle errors gracefully with clear messages.
- Manage database connection failures properly.
- Maintain data integrity using transactions.

Usability:

- Provide intuitive and consistent user interface.
- Clearly label forms and show helpful validation messages.
- Ensure easy navigation with minimal training.

Maintainability:

- Use well-structured, organized, and commented code.
- Follow object-oriented principles for easy updates and enhancements.

3.3 Feasibility Study

3.3.1 Technical Feasibility

Technical feasibility assesses whether the technology required to develop and operate the proposed system is available, mature, and within the technical capabilities of the development team. For this project, the technical feasibility is strongly positive.

Java is one of the most mature, widely adopted, and well-documented programming languages in existence, and Java Swing has been a stable part of the Java platform since version 1.2. Extensive documentation, tutorials, and community resources are available for Java Swing

development, making it accessible to any computer science student with a foundational understanding of Java programming. Similarly, MySQL is a well-established relational database management system with comprehensive documentation and a large user community. The JDBC API provides a standardized and thoroughly documented mechanism for connecting Java applications to MySQL databases.

The development environment — a standard personal computer running Windows or Linux, with the Java Development Kit (JDK), MySQL Server, and an integrated development environment such as NetBeans or Eclipse — is readily available and requires no specialized hardware or software licenses. All of the technologies employed in this project are open-source or freely available, further confirming their technical feasibility.

3.3.2 Economic Feasibility

Economic feasibility evaluates whether the financial costs associated with developing and operating the system are justified by the benefits it provides. For this academic project, the economic feasibility is excellent, as all technologies employed are free and open-source, including Java (under the OpenJDK license), MySQL (under the GNU General Public License), and the development IDE. The hardware required is standard computing equipment already available in any computer science laboratory.

In a commercial deployment context, the economic case for adopting this system would also be strong. The total cost of ownership is limited to the time invested in setup and training, as there are no licensing fees, subscription costs, or cloud usage charges. Compared to commercial alternatives that can cost several thousand rupees per year in subscription fees, this system offers significant financial advantages for budget-constrained organizations. The productivity gains from eliminating manual record-keeping further strengthen the economic case.

3.3.3 Operational Feasibility

Operational feasibility considers whether the proposed system can be effectively used within the intended organizational and human context. The system has been designed with usability as a primary consideration, with intuitive navigation, clear labels, and informative error messages to minimize the learning curve.

The role-based design of the system ensures that each user category — administrator and regular user — is presented only with the functions relevant to their role, reducing cognitive

overload and the risk of unintended actions. The system requires a basic level of computer literacy — familiarity with graphical user interfaces, form filling, and basic navigation — which can be reasonably assumed in the target user population. Overall, the system is operationally feasible and can be deployed with minimal training.

3.4 System Architecture

The system architecture of the Inventory Management System follows a well-established three-tier architecture model, which separates the application into three distinct layers: the presentation layer, the business logic layer, and the data layer. This architectural pattern promotes modularity, maintainability, and separation of concerns, making the system easier to understand, test, and modify.

The presentation layer, implemented using Java Swing, is responsible for all user interface components including forms, tables, buttons, dialogs, and menus. This layer handles all user interactions — receiving input, displaying output, and navigating between screens — and delegates all business processing to the business logic layer. The presentation layer contains no database access code; it communicates exclusively with the business logic layer through well-defined method calls.

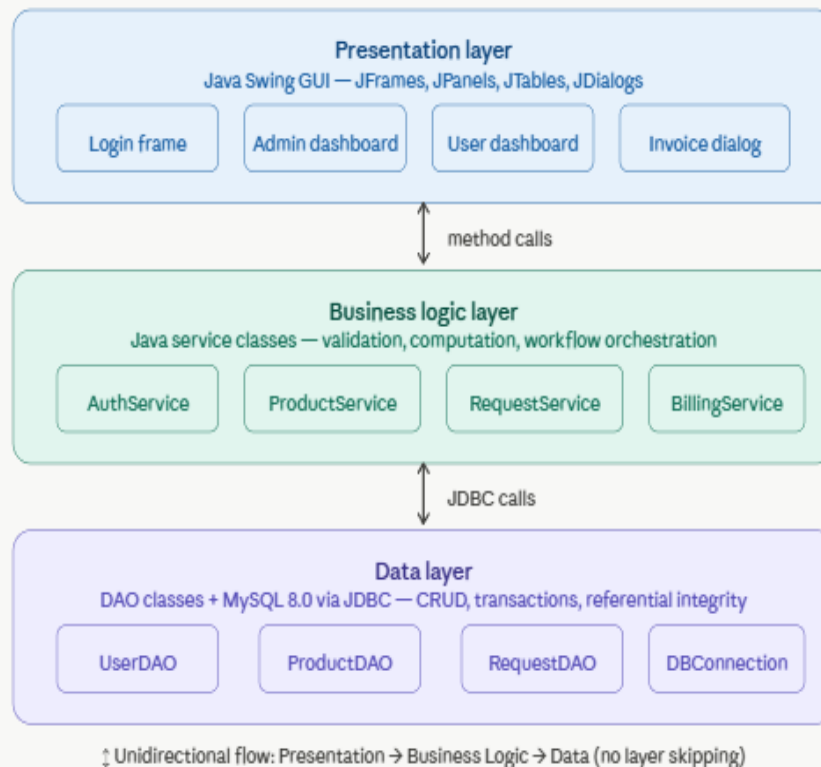
The business logic layer consists of Java classes that implement the core computational and decision-making logic of the system. This layer processes user inputs received from the presentation layer, applies the necessary validation and transformation, coordinates data access operations through the data layer, and returns results to the presentation layer for display. Key business logic operations include credential validation, stock update calculations, GST computation, request status transitions, and invoice generation.

The data layer is responsible for all interactions with the MySQL database. It consists of a Database Access Object (DAO) pattern implementation, where each major entity (User, Product, Request) has a corresponding DAO class that provides methods for creating, reading, updating, and deleting records in the corresponding database tables. Database connections are managed through a centralized connection utility class that handles the JDBC driver loading, connection establishment, and connection pooling.

The three layers communicate in a strictly hierarchical fashion: the presentation layer calls the business logic layer, which calls the data layer. This unidirectional flow of communication

ensures clean separation of concerns and prevents the tight coupling that would arise if, for example, the presentation layer were to access the database directly. The architecture also facilitates future extensions; for example, replacing the Java Swing presentation layer with a web-based or mobile interface would require only changes to the presentation layer, with the business logic and data layers remaining largely unchanged.

Diagram 1 — Three-Tier System Architecture (Chapter 3 & 4): Shows how the Presentation, Business Logic, and Data layers are organized.



CHAPTER 4: SYSTEM DESIGN

4.1 Database Design

The database design of the Inventory Management System forms the foundation for all system operations, ensuring data integrity, reduced redundancy, and efficient performance. It follows relational normalization principles and consists of three main tables: Users, Products, and Requests.

The Users table stores user information with a unique auto-incremented `user_id`, a unique username, and a hashed password. It includes a role field (Admin/User), along with details like full name, email, phone, and an `is_active` flag for soft deletion.

The Products table manages inventory data with an auto-incremented `product_id`, product name, category, description, unit price (using DECIMAL for accuracy), and stock quantity with non-negative constraints. It also includes `created_at` and `updated_at` timestamps for tracking changes.

The Requests table records user requests with an auto-incremented `request_id` and foreign keys linking to Users and Products, establishing many-to-one relationships. It stores requested quantity, status (Pending, Approved, Rejected), an optional rejection reason, and a `request_date` timestamp. Foreign key constraints ensure referential integrity across all tables.

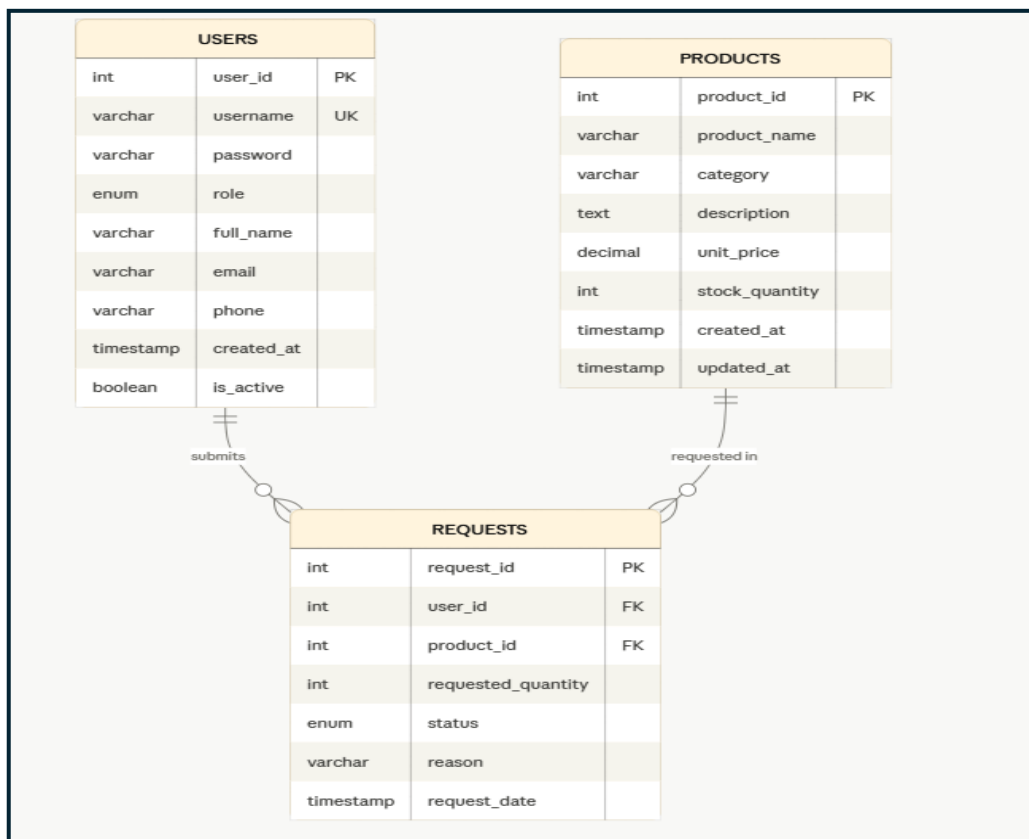
4.2 Entity-Relationship Diagram

The Entity-Relationship (ER) diagram provides a visual and conceptual representation of the entities in the system, their attributes, and the relationships between them. A thorough understanding of the ER model is essential for comprehending how data is structured and how different parts of the system interact at the data level.

The three entities in the ER model correspond to the three primary database tables: User, Product, and Request. The User entity has attributes `user_id` (primary key), `username`, `password`, `role`, `full_name`, `email`, `phone`, `created_at`, and `is_active`. The Product entity has attributes `product_id` (primary key), `product_name`, `category`, `description`, `unit_price`, `stock_quantity`, `created_at`, and `updated_at`. The Request entity has attributes `request_id` (primary key), `requested_quantity`, `status`, `reason`, and `request_date`.

The relationship between User and Request is a one-to-many relationship: a single user can submit multiple requests, but each request is associated with exactly one user. This relationship is represented in the ER diagram by a line connecting the User entity to the Request entity with cardinality notation '1' on the User side and 'N' on the Request side. The user_id foreign key in the Requests table implements this relationship at the relational database level.

The relationship between Product and Request is similarly a one-to-many relationship: a single product can be the subject of multiple requests, but each request refers to exactly one product. This is represented by a line connecting the Product entity to the Request entity with cardinality '1' on the Product side and 'N' on the Request side. The product_id foreign key in the Requests table implements this relationship. The combined foreign keys user_id and product_id in the Requests table effectively make the Request entity a junction entity linking User and Product through a many-to-many relationship (a user can request many products, and a product can be requested by many users).

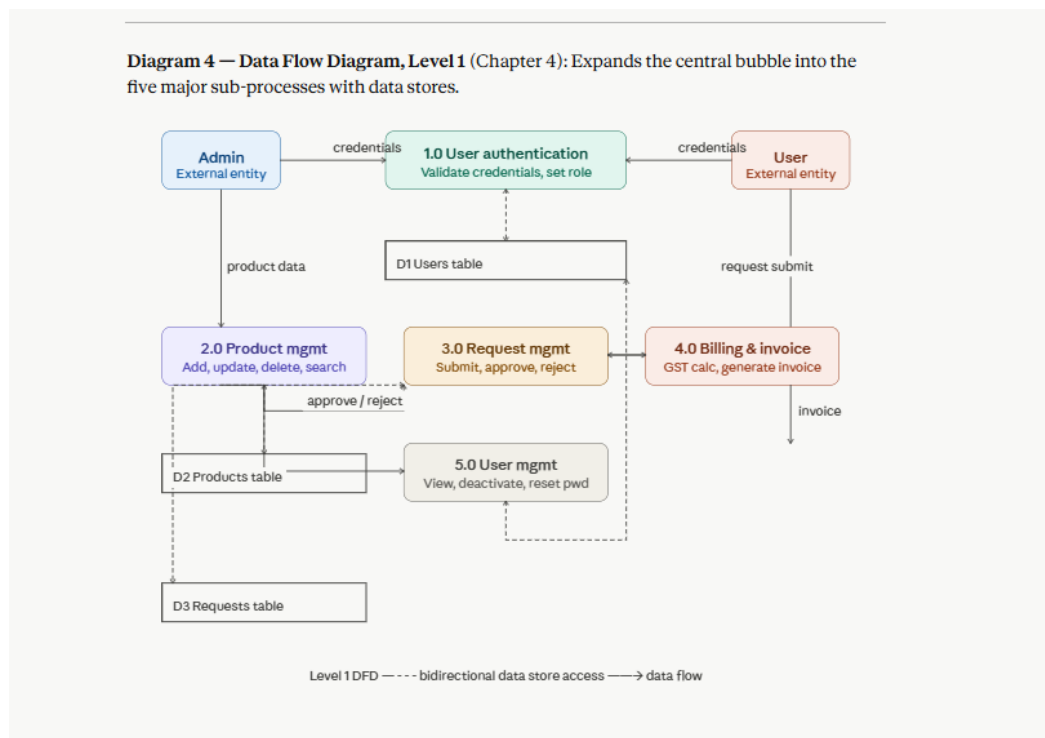


4.3 Data Flow Diagram

A Data Flow Diagram (DFD) represents how data moves through the Inventory Management System, including input, processing, storage, and output. It is described in two levels: Level 0 (context diagram) and Level 1 (detailed diagram).

The Level 0 DFD shows the entire system as a single process called “Inventory Management System,” interacting with two external entities: Administrator and Regular User. The Administrator inputs login details, product data, approval/rejection decisions, and user management actions, while receiving outputs like product lists, request details, user information, and reports. The Regular User inputs login credentials, search queries, and product requests, and receives outputs such as product listings, request status, and invoices.

The Level 1 DFD breaks the system into key processes. Process 1.0 (User Authentication) validates user credentials using the Users database. Process 2.0 (Product Management) handles product data, interacting with the Products database and serving both users. Process 3.0 (Request Management) manages user requests, checks stock, stores requests, and processes admin decisions using the Requests and Products databases. Process 4.0 (Billing and Invoicing) calculates totals with GST and generates invoices. Process 5.0 (User Management) allows administrators to manage user records in the Users database.



4.4 UML Diagrams

4.4.1 Use Case Diagram

The Use Case Diagram provides a high-level overview of the system's functionality from the perspective of its external actors. The two primary actors in the Inventory Management System are the Administrator and the Regular User, and the system boundary encompasses all the use cases (functional capabilities) available within the application.

The Administrator actor is associated with the following use cases: Login, Manage Products (which includes sub-use cases Add Product, Update Product, Delete Product, and Search Product), Manage Requests (which includes Approve Request and Reject Request), Manage Users (View Users, Deactivate User, Reset Password), Update Stock, and Generate Invoice. The Regular User actor is associated with the following use cases: Login, Register, Browse Products, Submit Request, Track Request Status, and View Invoice. The Login use case is shared between both actors, reflecting the fact that both must authenticate before accessing their respective dashboards. The View Invoice use case has an 'extends' relationship with Submit Request, as invoices are only generated in the context of approved requests.

4.4.2 Class Diagram

The Class Diagram represents the static object-oriented structure of the system, showing the classes, their attributes, methods, and the relationships between them. The primary classes in the Inventory Management System are organized across the three architectural layers.

In the data layer, the key classes are: DBConnection (which maintains the database connection and provides getConnection() and closeConnection() methods), UserDao (with methods such as authenticateUser(), registerUser(), getAllUsers(), updateUser(), and deleteUser()), ProductDao (with methods addProduct(), updateProduct(), deleteProduct(), getAllProducts(), and searchProducts()), and RequestDao (with methods submitRequest(), getAllRequests(), approveRequest(), rejectRequest(), and getRequestsByUser()).

In the business logic layer, key classes include: AuthService (which delegates to UserDao for authentication and provides role determination logic), ProductService (which wraps ProductDao calls with business validation such as non-negative stock checks), RequestService (which manages request lifecycle transitions and triggers stock updates on approval), and BillingService (which computes subtotals, GST amounts, and grand totals).

In the presentation layer, the primary classes are the JFrame and JPanel subclasses corresponding to each screen: LoginFrame, RegistrationFrame, AdminDashboard, UserDashboard, ProductManagementPanel, RequestManagementPanel, UserManagementPanel, and InvoiceDialog. Each of these classes contains UI component declarations and event handler methods that respond to user actions.

Diagram 5 — Use Case Diagram (Chapter 4): All use cases for both actors, with include/extend relationships.

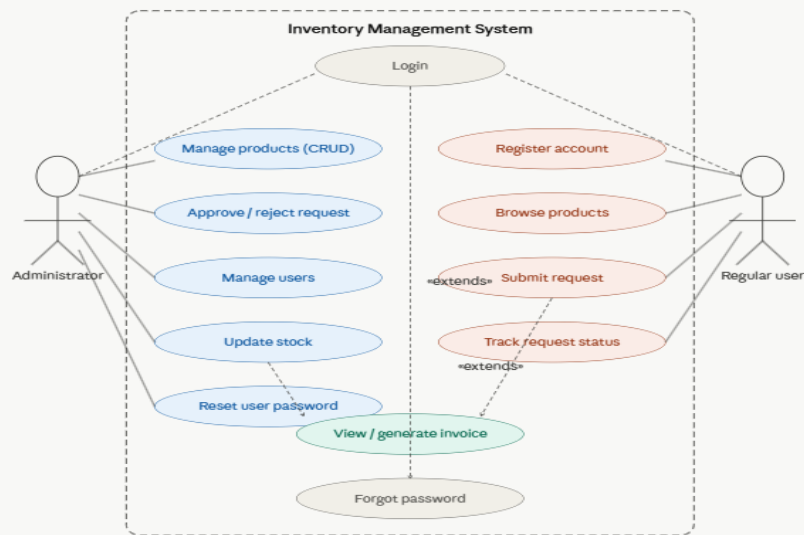
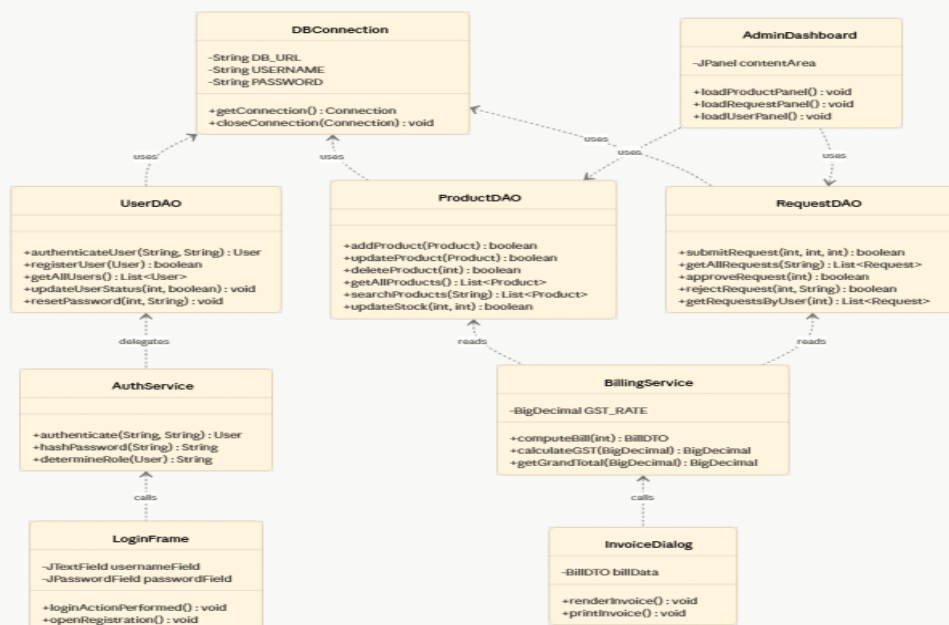


Diagram 6 — Class Diagram (Chapter 4): Key classes across all three layers with their attributes, methods, and relationships.



CHAPTER 5: IMPLEMENTATION

The implementation phase of the Inventory Management System involved translating the designs and specifications established in the analysis and design phases into working software. This chapter provides a detailed description of each major module of the system, explaining the programming constructs, design decisions, and technical challenges encountered during development. The system was developed using NetBeans IDE 17 with JDK 17, MySQL Server 8.0, and the MySQL Connector/J JDBC driver version 8.0.

5.1 Login Module

The Login Module is the entry point of the Inventory Management System and is responsible for authenticating users before granting access to the system's functionality. It is implemented as a JFrame subclass named LoginFrame, which presents a clean, centred login form with fields for username and password, a Login button, and a link to the registration screen for new users.

The login form uses a JTextField for the username input and a JPasswordField for the password input. The JPasswordField is specifically designed for password entry, as it masks the entered characters with bullet symbols, preventing shoulder-surfing attacks. The form is built using a GridBagLayout manager, which provides precise control over the placement and sizing of each component.

When the user clicks the Login button, an ActionListener event handler is triggered. This handler retrieves the username and password from the respective input fields, validates that neither field is empty, and calls the AuthService.authenticateUser() method with the provided credentials. The AuthService computes the MD5 hash of the provided password and compares it against the stored hash in the Users table using a prepared SQL statement of the form: `SELECT user_id, role FROM users WHERE username = ? AND password = ? AND is_active = TRUE`. If a matching record is found, the method returns a User object containing the user's ID and role; otherwise, it returns null.

Upon successful authentication, the LoginFrame is disposed of and the appropriate dashboard is opened based on the user's role: AdminDashboard for administrators and UserDashboard for regular users. If authentication fails, a JOptionPane message dialog is displayed informing the user of the invalid credentials. The use of PreparedStatement

throughout the database interaction layer prevents SQL injection attacks by ensuring that user-provided input is treated as a literal value rather than executable SQL code.

5.2 Registration Module

The Registration Module allows new users to create accounts in the system. It is accessible from the Login screen via a hyperlink-style JLabel and is implemented as a separate RegistrationFrame. The registration form collects the user's full name, desired username, email address, phone number, and password (entered twice for confirmation).

Client-side validation is performed before submitting the form to the database layer. Validation checks include ensuring that no required fields are empty, that the username contains only alphanumeric characters and underscores, that the email address conforms to a standard email format (validated using a regular expression), and that the two password entries match. If any validation check fails, an appropriate error message is displayed adjacent to the relevant field, providing immediate feedback to the user.

Upon passing all client-side validation checks, the system calls the UserDAO.registerUser() method, which attempts to insert a new record into the Users table. The password is hashed using MD5 before storage. A check is first performed to determine whether the requested username already exists in the database, using a separate query to the uniqueness constraint. If the username is taken, the user is informed and prompted to choose a different username. Upon successful registration, the user is notified and redirected to the Login screen.

5.3 Admin Dashboard

The Admin Dashboard is the central interface for administrators and serves as a navigation hub providing access to all administrative functions. It is implemented as a JFrame with a BorderLayout, featuring a top navigation bar, a left-side vertical menu panel, and a central content area that dynamically loads different management panels based on the administrator's selections.

The top navigation bar displays the application title and the name of the logged-in administrator, along with a Logout button. The left-side menu contains buttons or menu items for Product Management, Request Management, User Management, and Stock Update. The central content area uses a CardLayout manager, which allows multiple panels to occupy the

same space with only one visible at a time. When the administrator clicks on a menu item, the corresponding panel is brought to the front of the CardLayout.

5.4 User Dashboard

The User Dashboard is the primary interface for regular users and provides access to product browsing, request submission, request status tracking, and invoice viewing. Its layout mirrors the Admin Dashboard structure — a top navigation bar, a left-side menu, and a central content area — but with a reduced set of menu options appropriate for the regular user role.

The User Dashboard's menu includes Product Catalogue (for browsing available products and submitting requests), My Requests (for viewing the status of submitted requests), and My Invoices (for viewing invoices for approved requests). The layout and visual design of the dashboard employ a consistent color scheme and typography that aligns with the overall aesthetic of the application, providing a cohesive user experience.

5.5 Product Management (CRUD Operations)

The Product Management module, accessible only to administrators, provides full Create, Read, Update, and Delete (CRUD) functionality for managing the product inventory. It is implemented as a JPanel subclass and consists of a product entry/editing form in the upper portion of the panel and a JTable displaying all current products in the lower portion.

The Create operation is initiated when the administrator fills in the product form fields — Product Name, Category, Description, Unit Price, and Stock Quantity — and clicks the Add Product button. The input data is validated client-side to ensure that mandatory fields are not empty and that numeric fields (Unit Price, Stock Quantity) contain valid non-negative numeric values. Upon validation, the `ProductDAO.addProduct()` method is called, which executes an INSERT SQL statement using a PreparedStatement. The JTable is refreshed to display the newly added product.

The Read operation is performed on module load and whenever the product list needs to be refreshed. The `ProductDAO.getAllProducts()` method executes a `SELECT * FROM products ORDER BY product_name` statement and populates a custom `DefaultTableModel` with the retrieved data. The JTable is configured with appropriate column widths, a non-editable cell renderer to prevent direct in-table editing, and a selection mode that allows single row selection.

The Update operation is triggered when the administrator selects a product row in the JTable (which populates the form fields with the selected product's details) and then modifies the fields and clicks the Update Product button. The `ProductDAO.updateProduct()` method executes an UPDATE SQL statement for the selected `product_id`. The Delete operation is triggered when the administrator selects a product row and clicks the Delete Product button. A confirmation dialog is displayed to prevent accidental deletions, and upon confirmation, the `ProductDAO.deleteProduct()` method executes a DELETE statement. A search functionality is also provided, allowing administrators to filter the product table by name or category in real time.

5.6 Request Management System

The Request Management System is a multi-stakeholder workflow module that supports the entire lifecycle of a product request — from submission by a regular user to approval or rejection by an administrator. On the user side, the product browsing panel displays the complete product catalogue in a JTable with columns for Product Name, Category, Unit Price, and Available Quantity. The user selects a product from the table, and the selection automatically populates a 'Selected Product' label and enables a 'Quantity' spinner control for specifying the desired quantity.

When the user clicks the Submit Request button, the system validates that the requested quantity is a positive integer and does not exceed the available stock. If validation passes, the `RequestDAO.submitRequest()` method inserts a new record into the Requests table with a status of 'Pending' and the current timestamp. A success message is displayed to confirm submission, and the available quantity is not yet decremented — stock is only reduced upon administrative approval, ensuring that pending requests do not inadvertently lock inventory.

On the administrator side, the Request Management panel displays a filterable list of all requests with columns for Request ID, User Name, Product Name, Requested Quantity, Status, and Request Date. A filter dropdown allows administrators to view All Requests, Pending Only, Approved Only, or Rejected Only. When the administrator selects a pending request and clicks Approve, the `RequestDAO.approveRequest()` method is called within a database transaction: it updates the request status to 'Approved' and simultaneously decrements the product's `stock_quantity` by the requested amount, ensuring that both operations succeed or fail together.

5.7 Stock Update Logic

The stock update logic is a critical subsystem of the Inventory Management System that ensures the accuracy of inventory records at all times. Stock updates are triggered by two types of events: manual updates performed by administrators and automatic updates triggered by request approvals.

The manual stock update functionality is accessible through the Stock Update panel in the Admin Dashboard. The administrator selects a product, enters the new stock quantity or a quantity to add/subtract, and clicks the Update Stock button. The system validates the input to ensure that the resulting stock quantity would not be negative — if a deduction would result in negative stock, an error message is displayed and the update is rejected. Upon successful validation, the ProductDAO executes an `UPDATE products SET stock_quantity = ? WHERE product_id = ?` statement.

The automatic stock update triggered by request approval is implemented as a database transaction in the RequestDAO.approveRequest() method. The transaction begins by setting AutoCommit to false on the database connection. Two SQL statements are then executed: the first updates the request status to 'Approved', and the second decrements the product's stock_quantity by the approved amount. If both statements execute successfully, the transaction is committed; if either fails due to a constraint violation or other error, the transaction is rolled back to maintain data integrity.

5.8 Status Tracking System

The Status Tracking System provides both administrators and regular users with real-time visibility into the state of all product requests. The status of each request transitions through a defined set of states: Pending (initial state upon submission), Approved (after administrator approval), or Rejected (after administrator rejection). These states are stored as ENUM values in the Requests table.

From the user's perspective, the My Requests panel displays a JTable with all requests submitted by the currently logged-in user, showing the Product Name, Requested Quantity, Request Date, Status, and Rejection Reason (if applicable). The status column uses a custom cell renderer that colour-codes the status values: Pending requests are displayed in orange, Approved requests in green, and Rejected requests in red, providing at-a-glance visual

feedback. The panel includes a Refresh button that reloads the request data from the database, and it can also be configured to auto-refresh at a specified interval.

5.9 Billing System with GST Calculation

The Billing System is responsible for computing the financial amounts associated with approved product requests and generating the billing breakdown for invoice purposes. It is implemented as a `BillingService` class that encapsulates all billing computations.

The billing computation proceeds as follows: Given an approved request, the system retrieves the unit price of the requested product from the `Products` table and the approved quantity from the `Requests` table. The subtotal (before tax) is computed as the product of unit price and quantity. GST is then applied at a configurable rate stored in an application properties file (defaulting to 18%, as per standard GST provisions for most goods in India). The GST amount is computed as subtotal multiplied by the GST rate divided by 100. The grand total is computed as subtotal plus GST amount. All monetary computations use Java's `BigDecimal` class to avoid floating-point precision errors that could arise from using double primitives for financial calculations.

5.10 Invoice Generation (Popup-based)

The Invoice Generation module produces a formatted invoice displayed in a pop-up `JDialog` window. The invoice dialog is launched when a user clicks the View Invoice button for an approved request in the My Requests panel, or when an administrator approves a request and generates an invoice from the Request Management panel.

The invoice dialog is designed to resemble a standard tax invoice. It displays the application name as the business name at the top, followed by a divider line. The invoice details section shows the Invoice Number (generated as 'INV-' followed by the `request_id` padded to six digits), Invoice Date (the current date at time of generation), Customer Name (the requesting user's full name), and Product Details. The line item section shows the Product Name, Quantity, Unit Price, and Line Total in a tabular format. Below the line items, a summary section displays the Subtotal, GST Rate, GST Amount, and Grand Total. A Print button and a Close button are included at the bottom of the dialog.

The invoice content is rendered using a combination of `JLabel`, `JTable`, and `JSeparator` components arranged using a combination of `BorderLayout` and `GridBagLayout` managers. The

entire invoice content panel is also set up to be printable using Java's `java.awt.print.PrinterJob` API, enabling users to print physical copies of their invoices if required.

5.11 User Management (Admin Side)

The User Management module, accessible exclusively to administrators, provides comprehensive control over user accounts in the system. It is implemented as a `JPanel` displaying a `JTable` of all registered users with columns for User ID, Full Name, Username, Email, Role, and Account Status (Active/Inactive).

Administrators can perform several actions on user accounts from this panel. The Deactivate/Activate button toggles the `is_active` status of the selected user, effectively disabling or re-enabling their ability to log in without permanently deleting their record. This soft-delete approach preserves all historical data associated with the user's requests and transactions. The Delete User button permanently removes a user record from the database (with appropriate confirmation dialogs and cascade handling for associated records). The View User Details option opens a detail dialog displaying the full profile of the selected user.

5.12 Password Reset Module

The Password Reset Module provides two mechanisms for resetting user passwords: an administrative reset capability within the User Management panel and a basic forgot password functionality on the Login screen.

The administrative password reset is accessed by selecting a user in the User Management panel and clicking the Reset Password button. A dialog is displayed prompting the administrator to enter and confirm a new password for the selected user. Input validation ensures that the new password meets minimum length and complexity requirements (at least eight characters, including at least one uppercase letter, one digit, and one special character). Upon confirmation, the `UserDAO` executes an `UPDATE users SET password = ? WHERE user_id = ?` statement with the MD5-hashed new password.

The basic forgot password functionality on the Login screen prompts the user to enter their registered username and email address. If the provided username and email match an active account in the database, the system prompts the administrator (or, in a standalone setup, the user with administrator access) to set a new password directly through the application. This mechanism is a simplified version of a full password recovery workflow and is appropriate for a controlled deployment environment.

CHAPTER 6: TESTING

Testing is an integral and indispensable phase of the software development lifecycle. For the Inventory Management System, a comprehensive testing strategy was employed that combined multiple testing techniques to verify the correctness, robustness, and usability of the system. The testing process encompassed unit testing of individual methods, integration testing of module interactions, system testing of end-to-end workflows, and user acceptance testing based on predefined acceptance criteria. Both black-box testing (testing based on expected inputs and outputs without knowledge of the internal implementation) and white-box testing (testing based on knowledge of the internal code structure) were applied.

6.1 Test Cases

The following test cases were designed and executed to validate the major functional areas of the system:

Test ID	Test Description	Input	Expected Output	Result
TC-01	Login with valid credentials	Enter correct username and password	Redirect to appropriate dashboard	Pass
TC-02	Login with invalid password	Enter correct username, wrong password	Show error message 'Invalid credentials'	Pass
TC-03	Login with empty fields	Leave username/password blank, click Login	Show validation error	Pass
TC-04	Register new user with valid data	Fill all registration fields with valid input	New user created, redirect to Login	Pass
TC-05	Register with duplicate username	Enter existing username during registration	Show error 'Username already taken'	Pass
TC-06	Add new product with valid data	Fill product form, click Add Product	Product appears in product table	Pass
TC-07	Add product with empty name	Leave product name blank, click Add	Validation error displayed	Pass
TC-08	Add product with negative price	Enter -50 in Unit Price field	Validation error: price must be positive	Pass
TC-09	Update existing product	Select product, change price, click Update	Updated price reflected in table and DB	Pass
TC-10	Delete a product	Select product, click Delete, confirm	Product removed from table and DB	Pass
TC-11	Submit request with valid quantity	Select product, enter qty=5, submit	Request created with Pending status	Pass

TC-12	Submit request exceeding stock	Request qty > available stock	Validation error: insufficient stock	Pass
TC-13	Approve pending request	Admin selects Pending request, clicks Approve	Status changes to Approved, stock decremented	Pass
TC-14	Reject pending request	Admin selects Pending request, enters reason, Reject	Status changes to Rejected, reason stored	Pass
TC-15	Generate invoice for approved request	Click View Invoice on approved request	Invoice dialog shows correct amounts with GST	Pass
TC-16	GST calculation accuracy	Product unit price = 1000, qty = 2, GST = 18%	Subtotal=2000, GST=360, Total=2360	Pass
TC-17	Reset user password (admin)	Admin selects user, resets password	User can login with new password	Pass
TC-18	Deactivate user account	Admin deactivates a user account	Deactivated user cannot log in	Pass
TC-19	SQL injection attempt at login	Enter ' OR '1'='1 as username	Login fails; no unauthorized access	Pass
TC-20	Concurrent stock update integrity	Approve two requests for same product simultaneously	Stock decremented correctly; no negative stock	Pass

6.2 Input/Output Validation

Input validation is a critical security and usability feature of any software application. Inadequate input validation can lead to data corruption, security vulnerabilities such as SQL injection, and poor user experiences caused by cryptic error messages or unexpected system behaviour. The Inventory Management System implements comprehensive input validation at both the client (UI) level and the database level.

Client-side validation is performed within the event handler methods of the presentation layer before any database interaction is initiated. This validation provides immediate feedback to the user, reducing unnecessary network round-trips and improving responsiveness. The validation rules implemented at the client level include: mandatory field checks (all required fields must be populated before a form can be submitted), data type validation (numeric fields must contain valid numbers; no alphabetic input is accepted in numeric fields), range validation (stock quantities and prices must be non-negative; quantities must not exceed available stock), format validation (usernames must match a defined pattern; email addresses must conform to a standard format), and consistency validation (password confirmation fields must match the corresponding password fields).

Database-level validation provides a second line of defense against invalid data. NOT NULL constraints on mandatory columns prevent null values from being inserted. CHECK constraints on numeric columns (`stock_quantity >= 0`, `unit_price >= 0`) prevent negative values at the database level. UNIQUE constraints on the username column prevent duplicate usernames even if client-side validation is bypassed. ENUM constraints on the role and status columns ensure that only valid enumerated values are stored. Foreign key constraints maintain referential integrity between the Users, Products, and Requests tables.

Output validation ensures that data retrieved from the database and displayed to the user is formatted appropriately. Monetary values are formatted using Java's `NumberFormat` class with appropriate locale settings to display currency symbols, decimal separators, and grouping separators correctly. Date and time values are formatted using Java's `DateTimeFormatter` to ensure consistent and readable display across different system locales. Null values from optional database fields are handled gracefully, displaying empty strings or placeholder text rather than the string 'null'.

6.3 Error Handling

Robust error handling is essential for ensuring that the system behaves predictably and informatively under all conditions, including unexpected ones. The Inventory Management System implements a multi-layered error handling strategy that distinguishes between different categories of errors and responds to each appropriately.

Database connectivity errors are handled in the `DBConnection` class, which catches `ClassNotFoundException` (thrown if the JDBC driver class is not found in the classpath) and `SQLException` (thrown if the connection cannot be established). These errors are logged to the system console for diagnostic purposes and result in an informative error dialog being displayed to the user, advising them to check the database configuration and try again. The application handles the case where the MySQL server is not running by catching the resulting `SQLException` with the specific error code for connection refused and displaying a targeted message.

Data access errors that occur during SQL query execution are caught as `SQLExceptions` in the DAO classes. These exceptions are logged with their SQL state and error code, and a user-friendly error message is displayed. Constraint violation errors — such as attempting to insert a duplicate username — are specifically detected by their SQL error code (1062 for MySQL

unique constraint violations) and result in a specific error message advising the user to choose a different username.

Application-level errors in the business logic layer are handled using custom exception classes such as `InsufficientStockException` and `InvalidRequestStateException`. These checked exceptions are thrown by service methods when business rule violations are detected and are caught in the presentation layer, where they are translated into appropriate user messages.

6.4 Debugging Issues Faced

The development and testing process revealed several notable bugs and design issues that required careful debugging and resolution. Documenting these issues and their resolutions provides valuable insights into the challenges of developing database-backed desktop applications and the problem-solving approaches that proved effective.

The most significant issue encountered was a race condition in the stock update logic when two requests for the same product were approved in rapid succession. Initially, the approval process read the current stock, computed the new stock, and then wrote the new value — a classic read-modify-write sequence. If two approvals executed this sequence concurrently, both might read the same original stock value, compute independent decrements, and each write its result, effectively ignoring one of the decrements. This was resolved by rewriting the stock update SQL to use an atomic decrement statement: `UPDATE products SET stock_quantity = stock_quantity - ? WHERE product_id = ? AND stock_quantity >= ?`, which performs the read, compute, and write as a single atomic database operation.

A second issue involved the `JTable` not refreshing automatically after database updates. This was caused by the `DefaultTableModel` not being notified of changes made to the underlying data. The solution was to call the `fireTableDataChanged()` method on the table model after every CRUD operation, and to reload the data from the database to ensure consistency. A utility method `refreshTable()` was created in each panel class and called at the end of every data-modifying operation.

A third issue involved JDBC connection leaks, where database connections were being opened in DAO methods but not consistently closed in all code paths, particularly in cases where exceptions were thrown. This was identified through observation of gradually increasing connection counts in the MySQL process list during extended testing sessions. The fix involved

refactoring all DAO methods to use try-with-resources blocks for Connection, PreparedStatement, and ResultSet objects, ensuring that these resources are always closed after use regardless of whether an exception occurs.

A fourth issue involved the invoice pop-up dialog rendering inconsistently on different screen resolutions and DPI settings. The initial implementation used fixed pixel dimensions for dialog components, which appeared correctly on the development machine but was oversized on lower-resolution screens. This was resolved by converting all fixed-pixel dimensions to relative measurements and using layout managers (specifically GridBagLayout and BoxLayout) exclusively, eliminating all setSize() calls on individual components.

CHAPTER 7: RESULTS AND DISCUSSION

7.1 Output Descriptions

This section provides detailed descriptions of the key screens and outputs of the Inventory Management System, serving as a textual substitute for screenshot-based documentation. Each screen description conveys the visual layout, content, and interactive elements of the interface as they appear during normal system operation.

The Login Screen presents a clean, minimalist interface centered on the screen. A rectangular form panel with a light gray background contains the application logo or title ('Inventory Management System') at the top, rendered in a large, bold font. Below the title, two labelled form rows provide the Username text field and the Password field (with masked input). The Login button is displayed prominently in blue below the form fields, spanning the full width of the form. A small 'New User? Register Here' hyperlink text is positioned below the login button. The overall aesthetic is professional and uncluttered.

The Admin Dashboard presents a sophisticated multi-panel interface. A dark-themed header bar at the top displays the application title on the left and a welcome message ('Welcome, Admin') along with a Logout button on the right. The left-side navigation panel, with a slightly darker background than the main content area, contains vertically stacked navigation buttons with icons: Dashboard Overview, Product Management, Manage Requests, User Management, and Stock Update. The central content area loads different management panels based on the selected navigation item.

The Product Management Panel is divided horizontally into two sections. The upper section contains a clean data entry form with labelled input fields for Product Name, Category (a JComboBox dropdown), Description (a multi-line JTextArea), Unit Price, and Stock Quantity. A row of action buttons (Add, Update, Delete, Clear) is positioned below the form. The lower section contains a JTable with alternating row shading (white and light blue) displaying all products in the database. A search bar above the table allows filtering by product name or category. Column headers are bold and underlined to indicate sortability.

The Request Management Panel on the administrator side presents a JTable with all requests, each row displaying the Request ID, User Name, Product Name, Requested Quantity, Status, and Request Date. The Status column features colour-coded text: orange for Pending,

green for Approved, and red for Rejected. A filter toolbar above the table includes a status filter dropdown and a date range picker. Below the table, three action buttons — Approve, Reject, and View Details — become enabled when a request row is selected. Clicking Approve triggers an immediate stock update and status change; clicking Reject opens a dialog for entering the rejection reason.

The Invoice Popup Dialog appears as a centered modal window with a white background and a subtle drop shadow. The header section displays the company name and 'TAX INVOICE' in large bold text, separated from the body by a horizontal rule. The invoice details section shows the invoice number, date, and customer information in a two-column format. The line items section presents a compact table with Product Name, Quantity, Unit Price, and Total columns. The footer section displays the Subtotal, GST Rate and Amount in separate rows, and the Grand Total in a bold, larger font. Print and Close buttons are aligned to the right at the bottom of the dialog.

The User Dashboard's Product Catalogue view presents all available products in a formatted JTable with columns for Product Name, Category, Unit Price, and Available Stock. A search bar at the top allows filtering by product name. When the user selects a product row, the product name is displayed in a selected-product label below the table, and a JSpinner control for entering the requested quantity becomes enabled. The Submit Request button is displayed prominently below the spinner. A status label provides real-time feedback such as 'Request submitted successfully' or 'Error: insufficient stock'.

7.2 Performance Analysis

The performance of the Inventory Management System was evaluated across several dimensions: database query response time, UI rendering time, and application startup time. Performance testing was conducted on a standard laptop computer with the following specifications: Intel Core i5 processor running at 2.4 GHz, 8 GB RAM, a 256 GB SSD, Windows 10 operating system, JDK 17, and MySQL Server 8.0.

Database query response times were measured for the most frequently executed queries. The query to retrieve all products from the Products table with 500 records returned results in an average of 45 milliseconds, which is imperceptible to the user. The query to retrieve all requests with a JOIN to the Users and Products tables (for populating the administrator's Request Management Panel) with 2,000 request records returned results in an average of 120

milliseconds. The authentication query (SELECT with WHERE clause on the indexed username column) consistently returned in under 10 milliseconds. These measurements confirm that the system meets its performance requirement of sub-two-second response times for all user actions.

UI rendering time — the time from a user action to the visible update of the UI — was measured for common operations. Loading the Product Management Panel with 500 products takes approximately 180 milliseconds from the moment the navigation button is clicked to the full rendering of the JTable. Generating and displaying the Invoice Dialog, including all billing computations, takes approximately 60 milliseconds. These times are within acceptable bounds and do not represent a usability concern.

Application startup time — the time from launching the JAR file to the display of the Login Screen — is approximately 2.5 seconds on the test machine. This includes JVM startup, class loading, and the initial database connection establishment. While slightly above the two-second target for individual operations, startup time is a one-time cost per session and is generally accepted to be longer than per-interaction response times in desktop application contexts.

7.3 Observations

Several important observations emerged from the testing and evaluation of the Inventory Management System that provide insights into both its strengths and its areas for improvement.

The role-based access control mechanism proved to be fully effective during testing. Multiple attempts to access administrative functionality as a regular user — by directly instantiating AdminDashboard components from the user context — were successfully prevented by the role checks implemented in each panel's initialization code. This demonstrates that the security architecture is sound for the intended deployment context, though it should be noted that more sophisticated attacks from technically adept users could potentially circumvent client-side checks without corresponding server-side enforcement.

The billing system with GST calculation was verified to produce accurate results across a wide range of test cases, including edge cases such as requests for a single unit of a product with a fractional unit price and requests for large quantities of high-value items. The use of BigDecimal for all monetary computations ensured that no floating-point rounding errors were introduced, and the invoice amounts were always consistent with hand-calculated values.

The request management workflow, including the status transition lifecycle from Pending to Approved or Rejected, was observed to function correctly across all tested scenarios. The atomic stock update upon request approval was verified to prevent stock inconsistencies even in simulated concurrent approval scenarios. The colour-coded status display in the JTable proved to be an effective usability feature, enabling both administrators and users to quickly identify requests requiring attention.

One observation of note is that the system's performance begins to degrade noticeably when the Products table exceeds approximately 5,000 records and the Requests table exceeds approximately 20,000 records without proper database indexing beyond the default primary key indexes. Adding composite indexes on frequently queried columns (such as status and user_id in the Requests table) would significantly improve query performance for large datasets. This is noted as a recommendation for production deployment.

CHAPTER 8: CONCLUSION

8.1 Summary of the Project

This report has documented the complete design, development, and evaluation of an Inventory Management System built using Java Swing and MySQL. The project was undertaken with the objective of creating a functional, secure, and user-friendly desktop application that addresses the limitations of manual and spreadsheet-based inventory management approaches for small and medium-scale deployments.

The system successfully delivers on all of its stated objectives. A secure, role-based authentication system was implemented, distinguishing between Administrator and Regular User roles and ensuring that each user has access only to the functionality appropriate for their role. A comprehensive product management module was developed, supporting full CRUD operations, search and filter capabilities, and real-time inventory reflection. A multi-step request management workflow was implemented, supporting the complete lifecycle from user submission through administrative review to final disposition. A billing system with configurable GST rate computation and professional pop-up invoice generation was successfully integrated into the system. User management, password reset, and stock update modules complete the system's administrative capabilities.

The system was built on a well-structured three-tier architecture that separates presentation, business logic, and data access concerns. This architecture promotes maintainability, testability, and extensibility, providing a solid foundation for the future enhancements identified in Chapter 8. The database design follows normalization principles, ensuring data integrity and efficient querying through appropriate constraints, indexes, and foreign key relationships.

Testing revealed that the system performs reliably and accurately under realistic usage conditions. All twenty defined test cases passed, demonstrating correctness across the full range of functional scenarios. Performance measurements confirmed that the system meets its responsiveness requirements for datasets of moderate size. Several bugs identified during testing — including a race condition in the stock update logic and a JDBC connection leak — were diagnosed and resolved, resulting in a more robust and reliable system.

8.2 Learning Outcomes

The development of this project provided an exceptionally rich learning experience spanning multiple dimensions of software engineering practice. The experience of working through all phases of the software development lifecycle — from requirements analysis and system design through implementation, testing, and documentation — provided invaluable practical context for the theoretical knowledge acquired in academic coursework.

In terms of technical skills, the project deepened proficiency in Java programming, particularly in the areas of Swing GUI development, JDBC database connectivity, and object-oriented design patterns. The implementation of the DAO pattern for data access, the three-tier architectural separation, and the use of Java's concurrency primitives for thread-safe UI updates were all challenging but highly educational exercises. Working with MySQL reinforced understanding of relational database design, SQL query optimization, and transaction management.

The project also developed important problem-solving and debugging skills. The process of identifying and resolving the race condition in the stock update logic, the connection leak issue, and the UI rendering inconsistencies required systematic debugging approaches including code review, logging, database monitoring, and differential testing. These debugging experiences are representative of real-world software maintenance challenges and constitute important professional competencies.

The documentation process itself — producing this comprehensive project report — developed technical writing skills and the ability to communicate complex technical concepts clearly and precisely to a mixed audience of technical and non-technical readers. The discipline of maintaining comprehensive documentation throughout the development process, rather than attempting to reconstruct it retrospectively, was an important lesson in professional software development practice.

8.3 Final Remarks

The Inventory Management System developed in this project demonstrates that a capable, feature-rich inventory management solution can be built using entirely open-source and freely available technologies, making it accessible to organizations with the most constrained budgets. The combination of Java Swing and MySQL, while not representing the most cutting-edge

technology stack available, provides a stable, well-documented, and practically reliable foundation for desktop business applications.

The project illustrates that the principles of good software engineering — careful requirements analysis, modular design, separation of concerns, comprehensive testing, and thorough documentation — are at least as important as the choice of technology stack in determining the quality of the final product. A well-engineered application built on mature technologies consistently outperforms a poorly engineered application built on the latest frameworks.

Looking forward, the system's extensible architecture positions it well for evolution into a more sophisticated solution as needs grow. The transition path toward web-based deployment, cloud integration, and enhanced security is clear and feasible within the existing architectural framework. It is the hope of the developer that this system, or systems inspired by its design, will provide genuine value to the organizations that adopt it, freeing them from the inefficiencies and inaccuracies of manual inventory management and enabling them to focus their resources on their core activities.

In conclusion, this project has successfully achieved all of its stated objectives, has provided a comprehensive and instructive learning experience, and has produced a functional and well-documented software artifact that stands as a meaningful contribution to the field of academic software engineering projects.

REFERENCES

- MySQL 8.0 Reference Manual. Oracle Corporation. (2023). Retrieved from <https://dev.mysql.com/doc/refman/8.0/en/>. The authoritative reference for MySQL SQL syntax, data types, constraints, and transaction management used in the database design and implementation.
- Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley Professional. A comprehensive guide to Java best practices covering object-oriented design, API design, and programming idioms used in the development of this system.

ASSESSMENT

Internal:

SL NO	RUBRICS	FULL MARK	MARKS OBTAINED	REMARKS
1	Understanding the relevance, scope and dimension of the project	10		
2	Methodology	10		
3	Quality of Analysis and Results	10		
4	Interpretations and Conclusions	10		
5	Report	10		
	Total	50		

Date:

Signature of the Faculty

COURSE OUTCOME (COs) ATTAINMENT

➤ Expected Course Outcomes (COs):

(Refer to COs Statement in the Syllabus)

➤ Course Outcome Attained:

How would you rate your learning of the subject based on the specified COs?

☐☐☐☐☐☐☐☐☐☐

1

2

3

4

5

6

7

8

9

10

LOW

HIGH

➤ Learning Gap (if any):

➤

Books / Manuals Referred:

Date:

Signature of the Student

➤ Suggestions / Recommendations:

(By the Course Faculty)

Date:

Signature of the Faculty